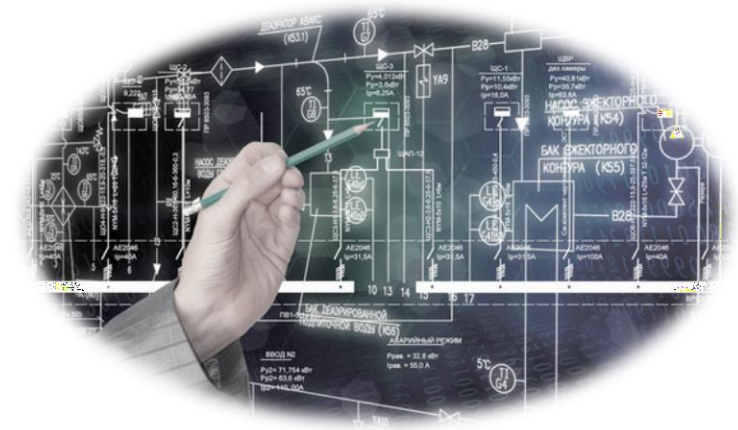


Neural Networks

Convolutional Neural Networks



* 교재 슬라이드를 수정하여 작성됨

Fashion-MNIST (another image-based classification dataset)



```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

```
# keras의 데이터셋의 패션 MNIST 데이터를 학습용, 테스트 데이터로 구분
fashion_mnist = keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) =\
fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/train-labels-idx1-ubyte.gz
```

```
32768/29515 [=====] - 0s 0us/step
```

```
...
```

DenseNet for F-MNIST



```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

train_images, test_images = train_images / 255, test_images / 255
model.fit(train_images, train_labels, epochs=10)

test_loss, test_acc = model.evaluate(test_images, test_labels,\
                                     verbose=2)

print('\n테스트 정확도:', test_acc)
```

```
Epoch 1/10
1875/1875 [=====] - 7s 4ms/step - loss: 0.4867 -
accuracy: 0.8246
... <중간생략> ...
313/313 - 1s - loss: 55.0649 - accuracy: 0.8592

테스트 정확도: 0.8592000007629395
```

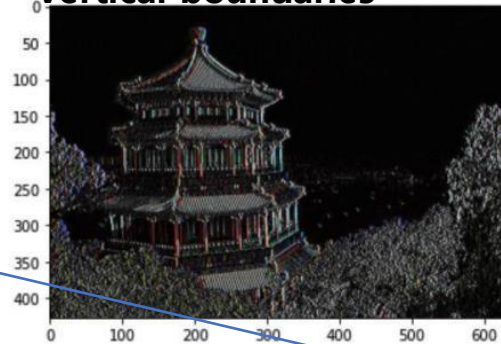
How to improve the model?

Receptive Fields and Convolutional Neural Networks

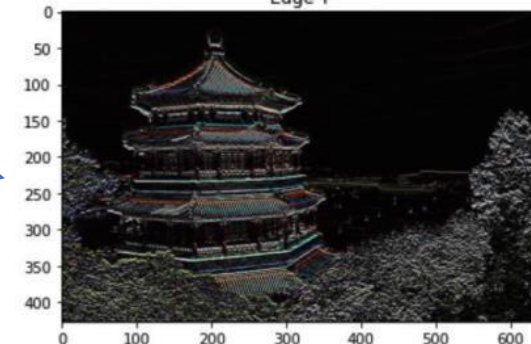
- When visual information comes in, we know that neurons in the visual cortex are activated for a region within a portion of the visual field to receive the information.
- The area of visual information that influences the activation of a single neuron is called the receptive field.
- For example, some neurons respond to images of horizontal lines, some respond to images of vertical lines, and a larger receptive field that combines responses to these low-level images results in a response to the pattern



Filter to emphasize
vertical boundaries



Filter to emphasize
horizontal boundaries



Receptive Fields and Convolutional Neural Networks

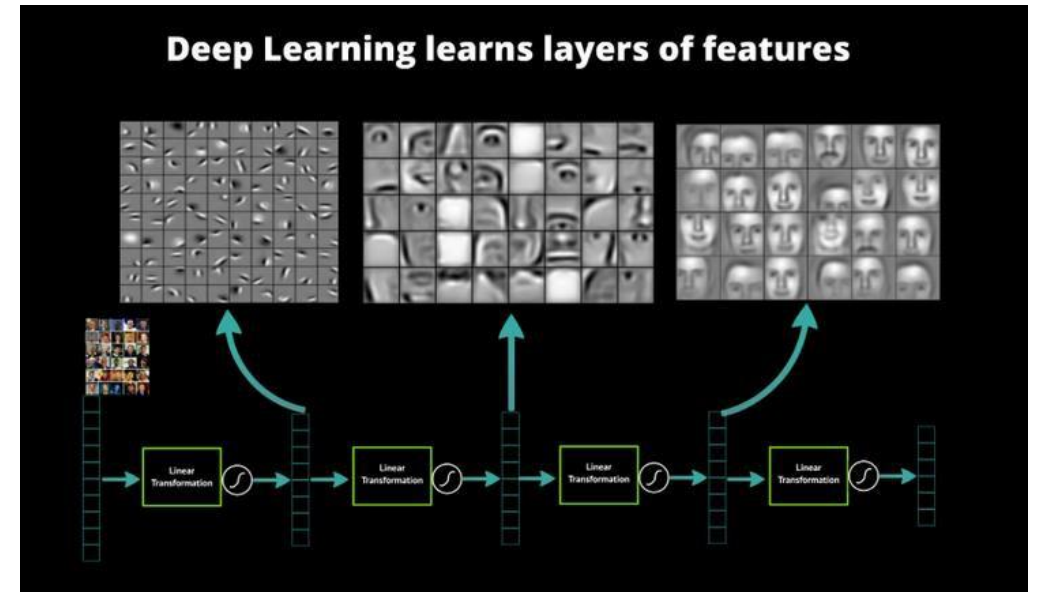
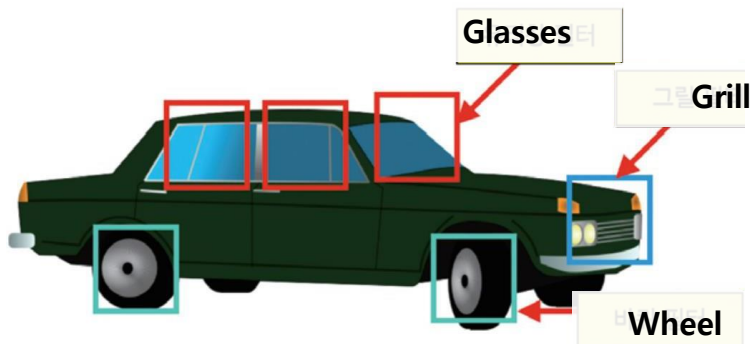
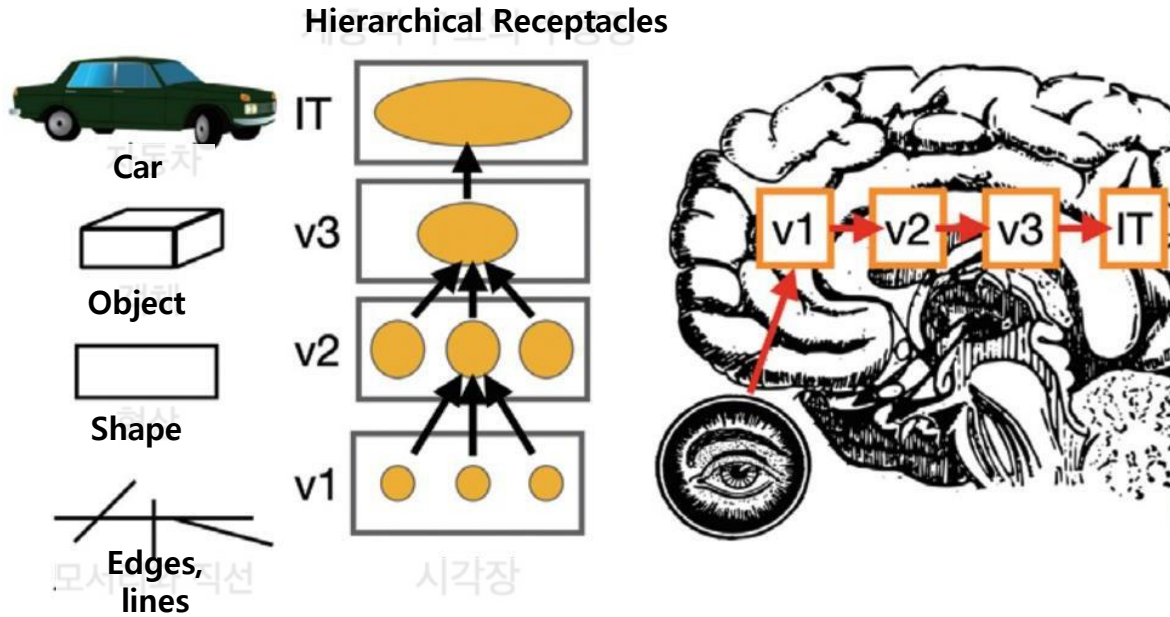


Image Filters

- Image Filters Overview: Image filters, also known as kernels, are core tools in digital image processing. They are used to enhance images or extract information by applying various effects such as edge detection, blurring, and sharpening.
- How Image Filters Work: An image filter is a small matrix that modifies the image by applying a convolution operation. This involves overlaying the filter on the image, multiplying its values by the original pixel values, and summing the result to produce a new pixel value.
- Common Uses: Filters can highlight edges, reduce noise, or blur images to remove detail. For example, an edge detection filter highlights the boundaries of objects, while a blur filter can reduce image noise.
- Custom vs. Learned Filters: Traditionally, image filters were hand-designed, with specific patterns to achieve desired effects. However, a significant advancement in image processing has been the development of systems that can learn the most effective filters for a given task.



Fig. 3: original image

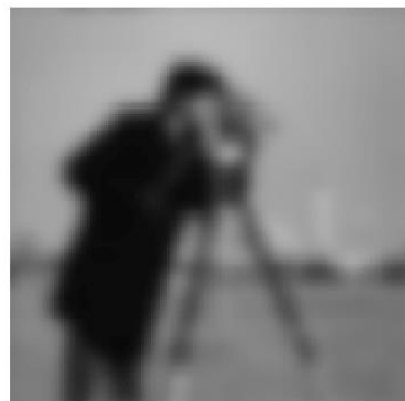


Fig. 4: Result of Gaussian low pass filter



Fig. 5: Gaussian high pass filter

SIFT and the Evolution to CNN – From handcrafted filters to learned features

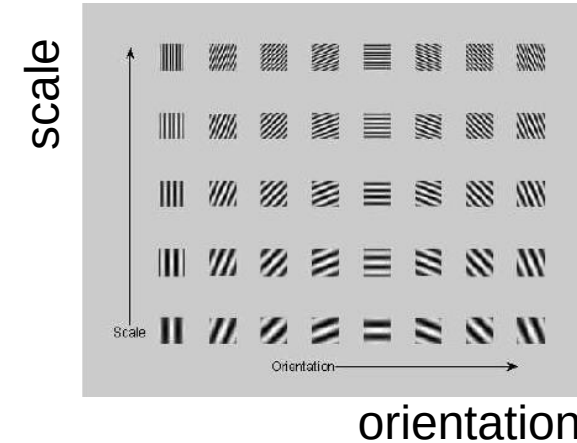
“If we can describe an image well using a set of filters, we can capture its distinctive properties.”

- Image analysis starts with identifying what features matter.
- Early methods relied on handcrafted filters to extract meaningful patterns.
- But how do we design these filters? → “Experts had to craft them manually... one by one.”

SIFT = Scale-Invariant Feature Transform

- A landmark method in traditional computer vision.
- Extracts keypoints that are:
 - Scale-invariant
 - Rotation-invariant
 - Robust to illumination changes

SIFT is manual design! Filters are fixed, not learned from data

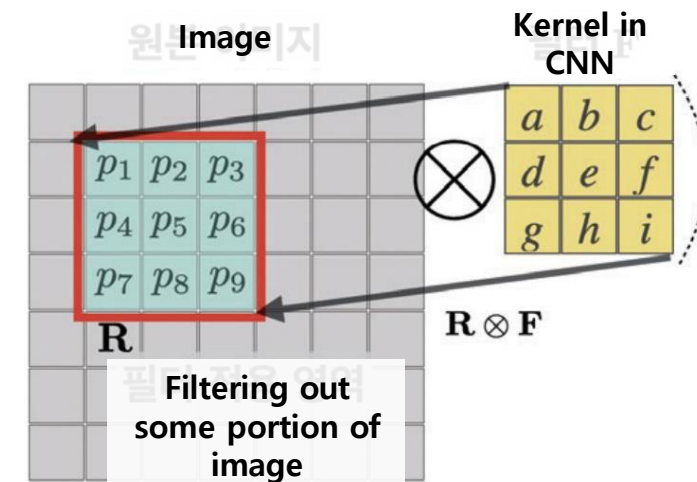
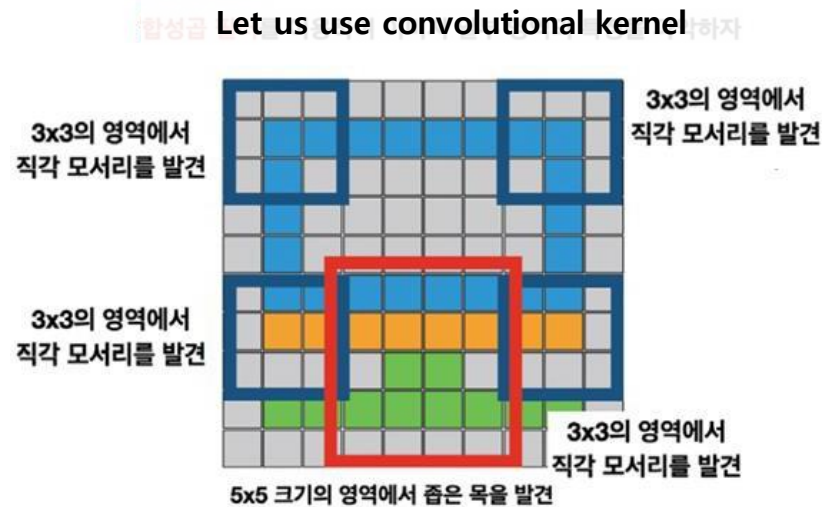
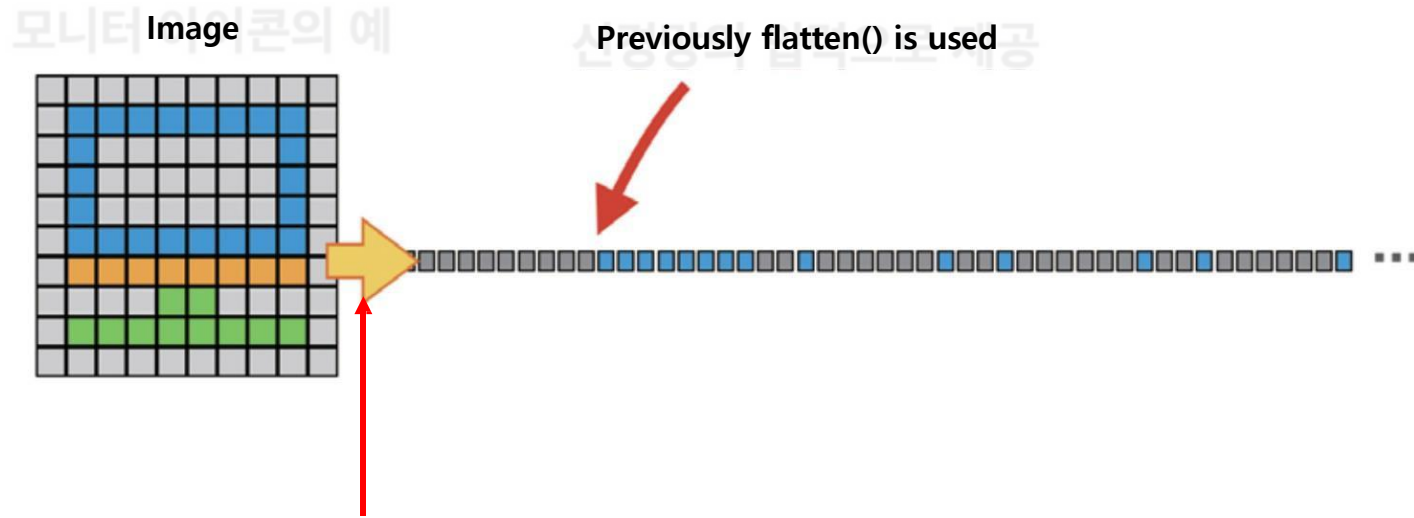


Key points computed by SIFT

Learnable filters for image analysis?

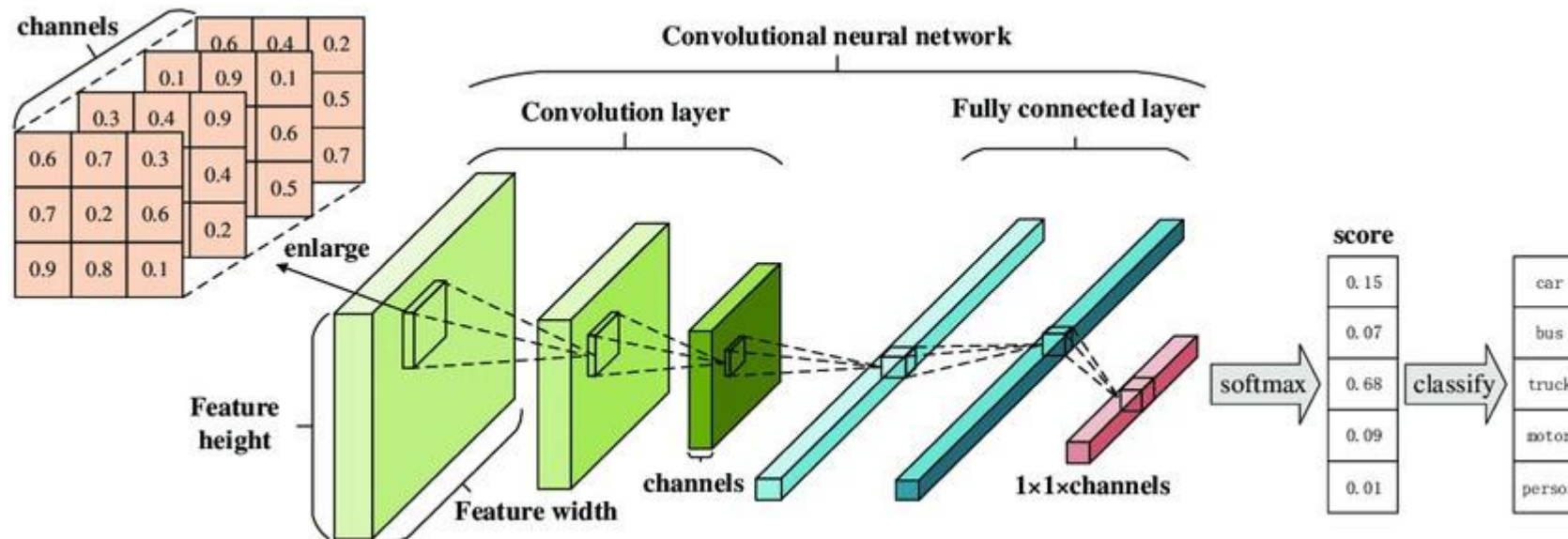
- In the context of Convolutional Neural Networks (CNNs), image filters (also known as kernels) are small matrices used to apply effects like edge detection, blurring, sharpening, and more to images.
- How They Work: These filters are applied to the original image through a process called convolution. By sliding the filter over the image and computing the dot product at each position, new feature maps that highlight specific attributes of the image are created.
- Role in CNNs: Filters in CNNs automatically learn to detect important features during training, such as edges, textures, or shapes, which are critical for understanding and analyzing images.
- Feature Extraction: As the image passes through successive convolutional layers in a CNN, filters can capture increasingly complex aspects of the image. The network learns which filters are most effective for the specific tasks (e.g., classification, recognition) it's being trained on.

Filters and Convolutional Neural Networks



Convolutional Neural Networks Architecture

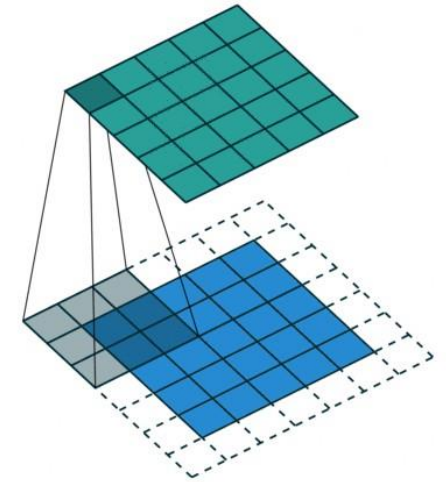
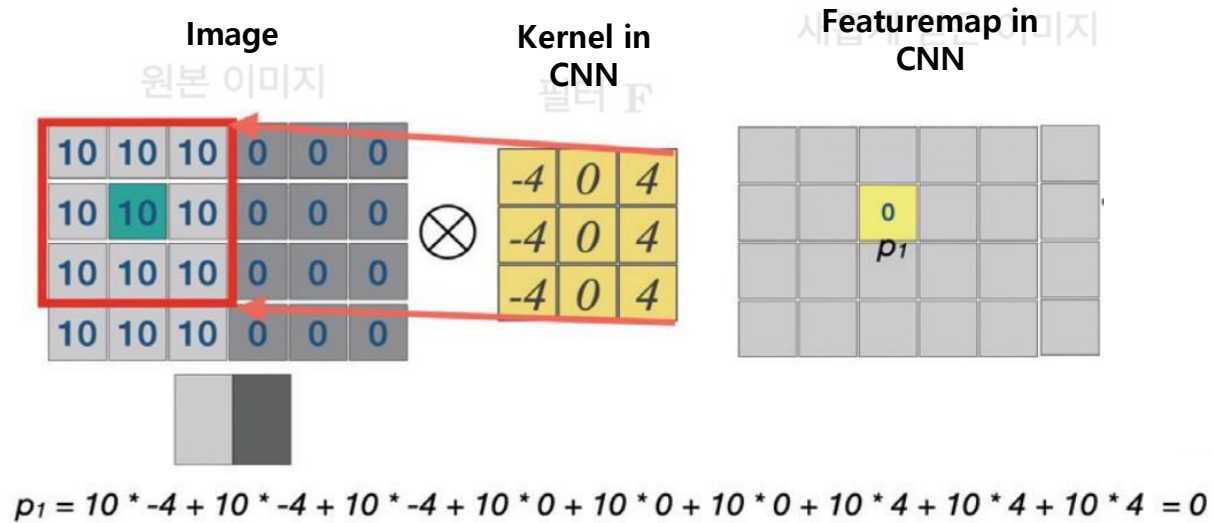
- CNNs excel in feature extraction for image classification by learning to emphasize the most relevant features through their unique architecture and training process.
- The optimization of filter weights is a central aspect of training CNNs, enabling them to learn powerful feature representations directly from the data.
- Understanding the process of weight optimization and feature extraction is key to leveraging the full power of CNNs for a wide range of image-based tasks.



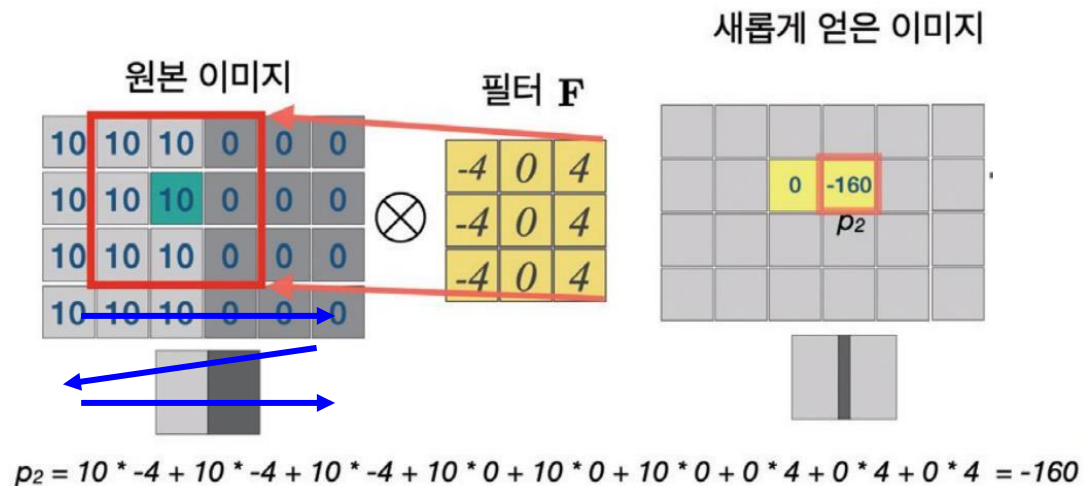
Composition of a Convolutional Layer

- **Filters/Kernels:** Small, learnable matrices that scan over the input to extract features. Each filter detects different aspects of the input (e.g., edges, colors, gradients).
- **Stride:** The number of pixels by which the filter moves across the input. A stride of 1 moves the filter one pixel at a time.
- **Padding:** Adding zeros or copying edge values around the input image to preserve the spatial size of the output feature map.
- **Feature Map:** The output generated by a filter over the input is called a feature map. It represents the detected features of the input.
- **Activation Function:** Often, a non-linear function like ReLU is applied to the feature map to introduce non-linearity, allowing for the detection of complex patterns.

Filters and Convolutional Neural Networks



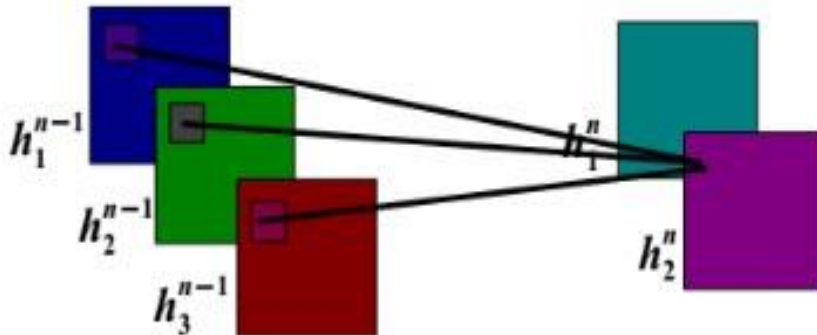
Apply the filter to the entire image, starting at the top left of the original image and moving to left and bottom.



Convolutional Layer

$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{kj}^n)$$

output feature map input feature map kernel



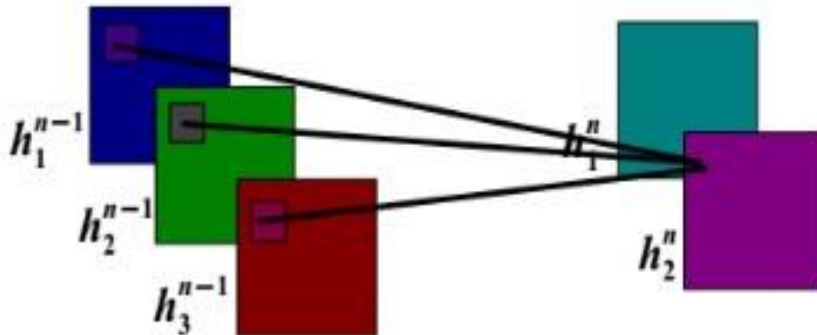
When multiple channels are input into a convolutional layer, the convolution operation processes these channels in a specific way to produce the output feature map. Here's a step-by-step explanation:

1. **Input Feature Maps:** The input to the convolutional layer consists of multiple channels. For example, an RGB image has three channels: Red, Green, and Blue.
2. **Kernel:** The convolutional layer uses a set of filters (kernels). Each filter has the same depth as the number of input channels. For example, if there are three input channels, each filter will also have three channels.

Convolutional Layer

$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{kj}^n)$$

output feature map input feature map kernel



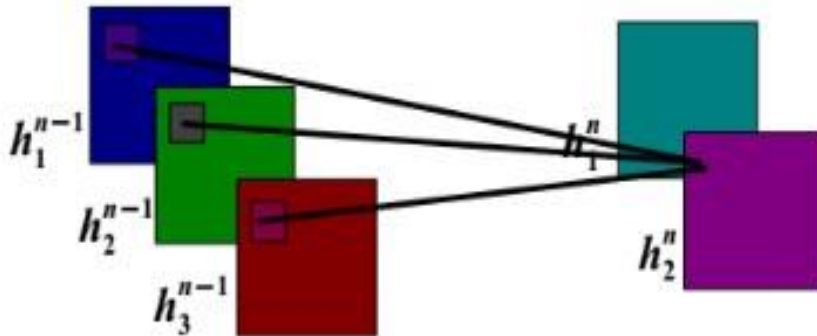
3. Convolution Operation:

- Each filter slides (convolves) over the input feature maps.
- For each position, element-wise multiplication is performed between the filter and the corresponding region of the input feature maps.
- The results of these multiplications are summed up to produce a single value. This operation is repeated for every spatial position in the input feature maps, resulting in a single 2D feature map.

Convolutional Layer

$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{kj}^n)$$

output feature map input feature map kernel



4. Summing Across Channels:

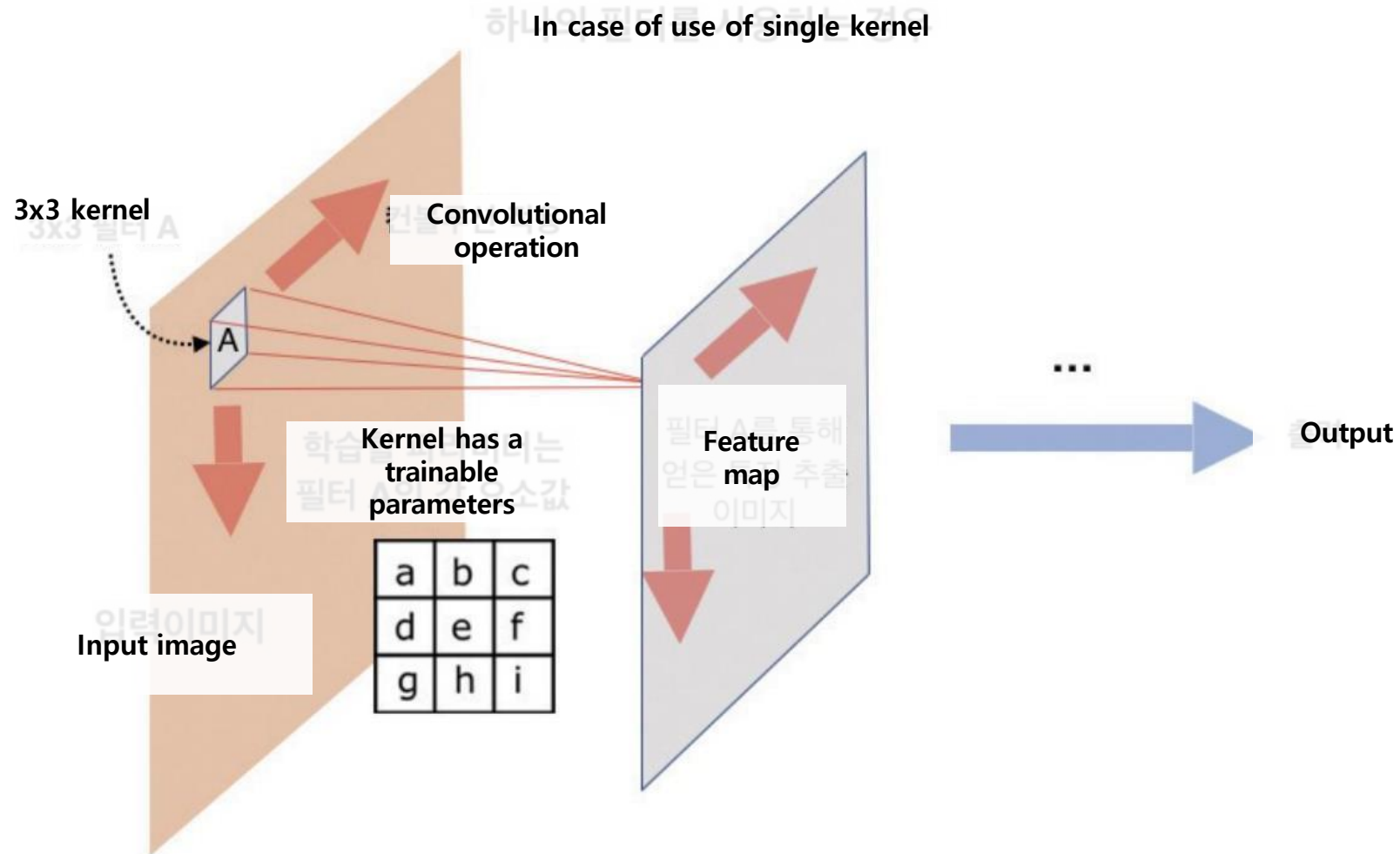
- Since each filter spans all input channels, the convolution operation involves summing the results across all channels.
- Specifically, for each position, the element-wise multiplications and summing are performed separately for each channel and then summed across all channels to obtain a single output value for that position.

5. Bias and Activation Function:

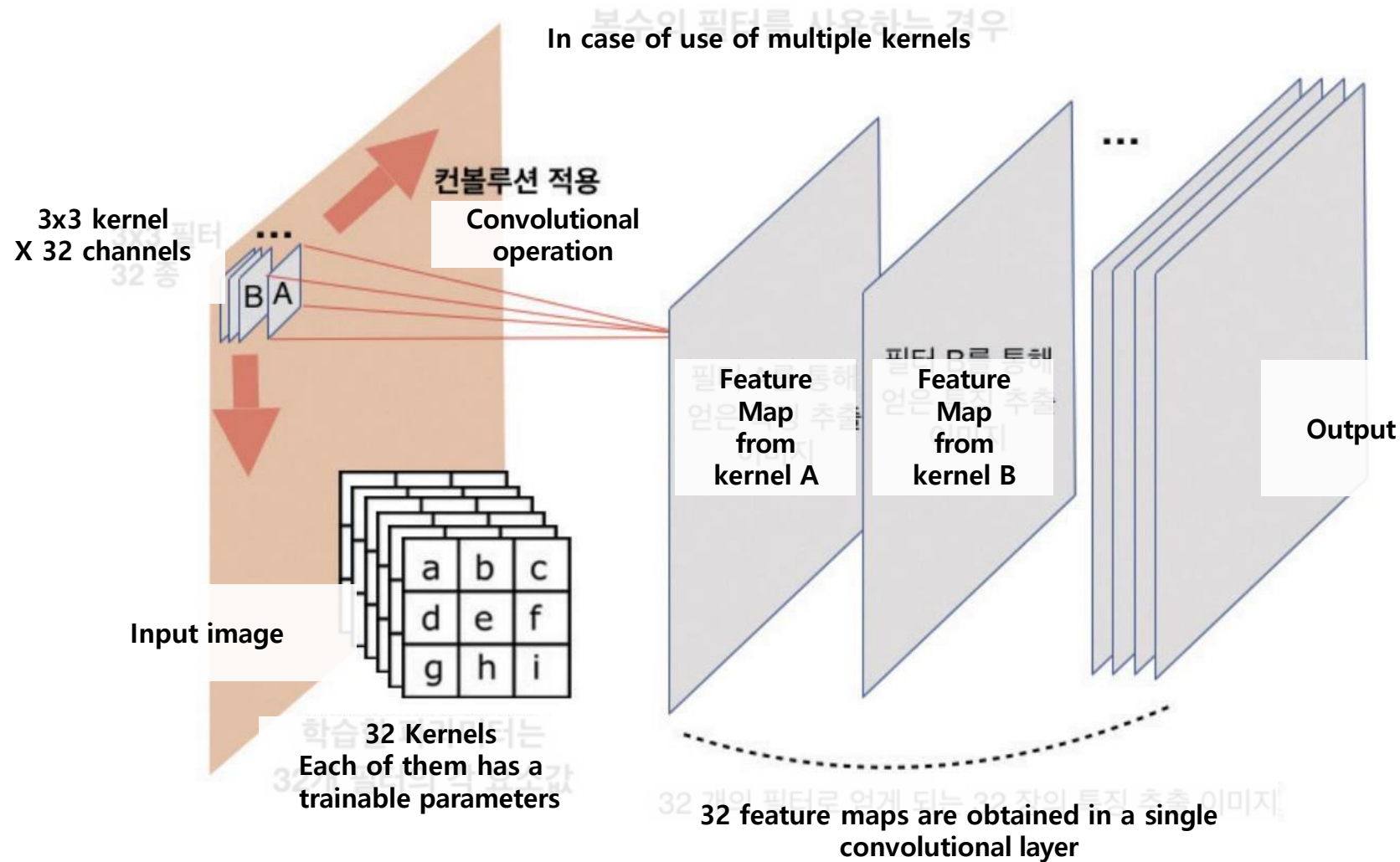
After the convolution operation, a bias term is added to each element of the resulting feature map.

6. Output Feature Map: The process is repeated for each filter in the convolutional layer. If the layer has N filters, it produces N output feature maps, each corresponding to a filter.

Filters and Convolutional Neural Networks

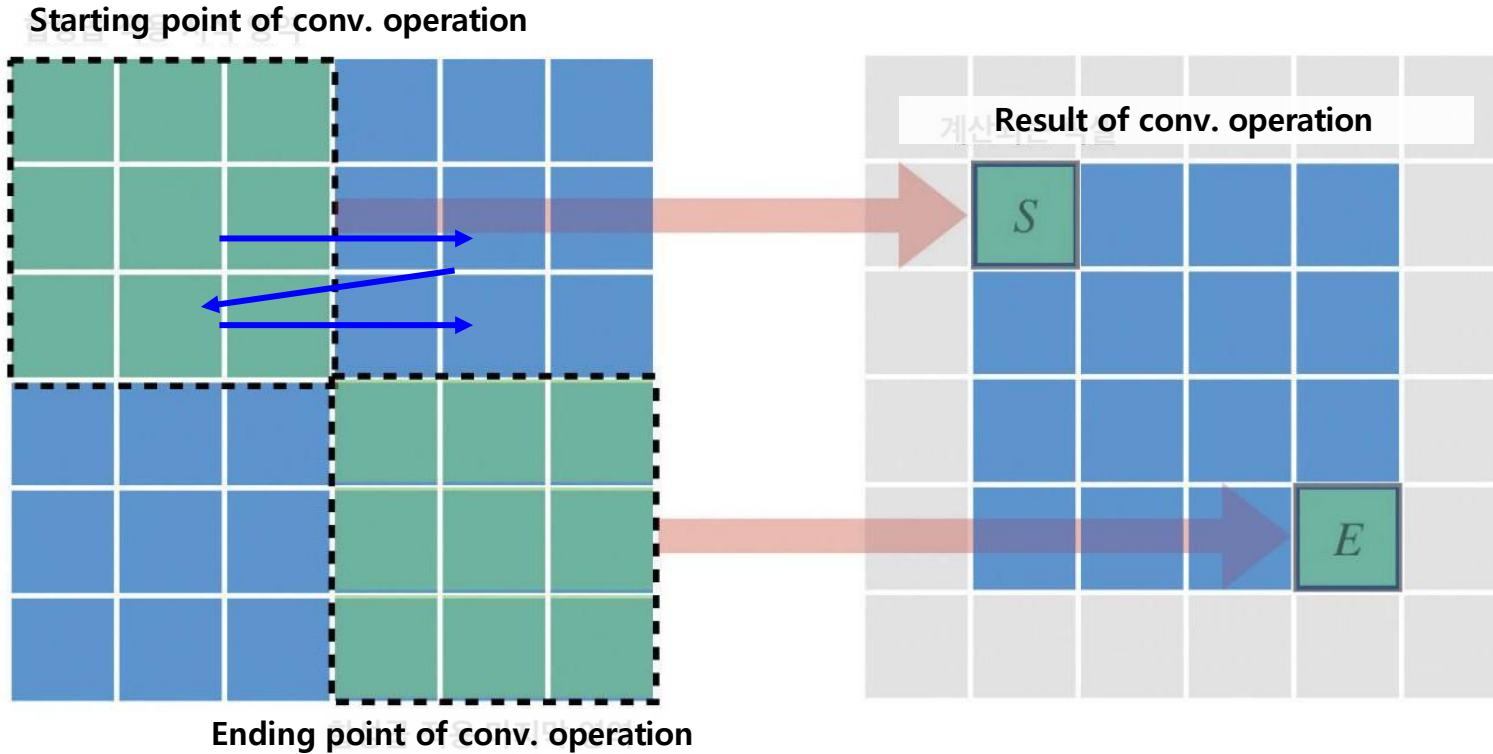


Filters and Convolutional Neural Networks

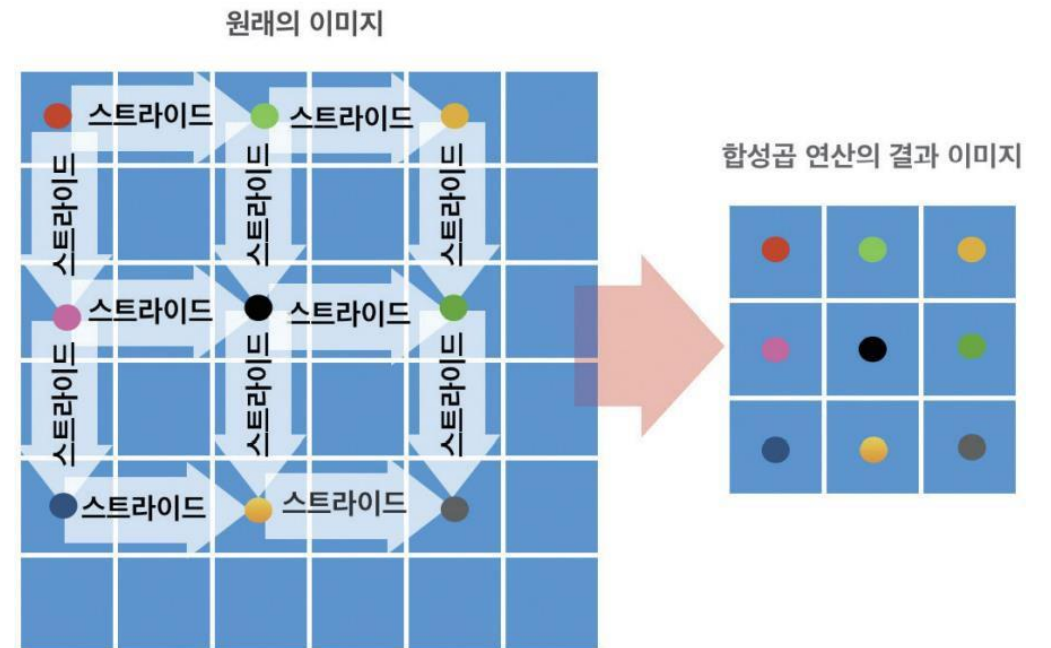
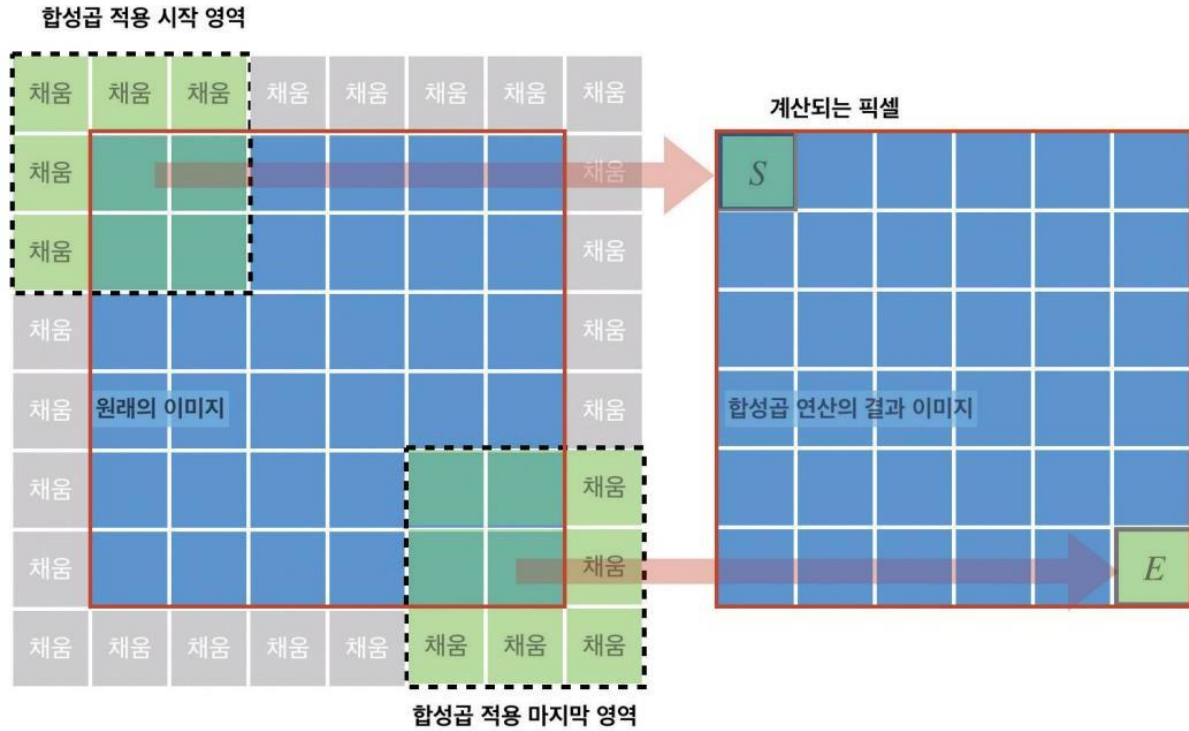


Filters and Convolutional Neural Networks

- Extracting features of images (feature map)

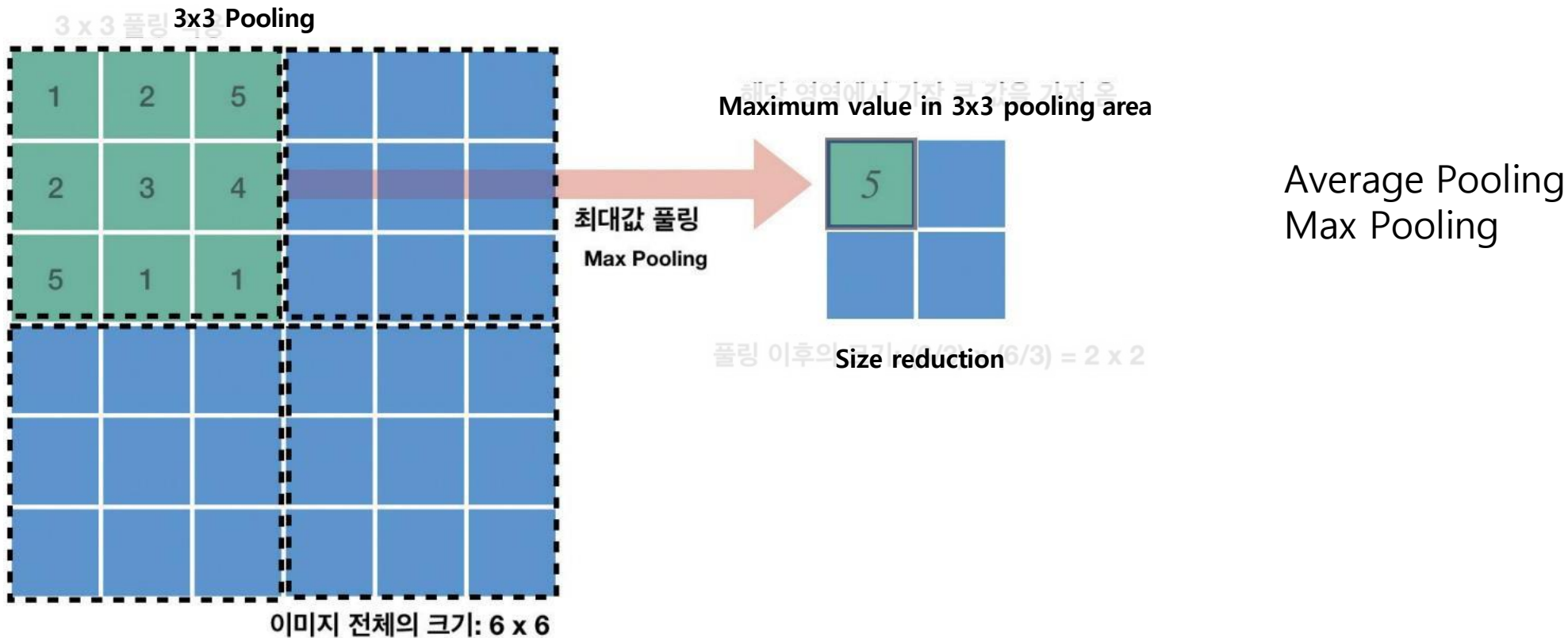


Padding and stride

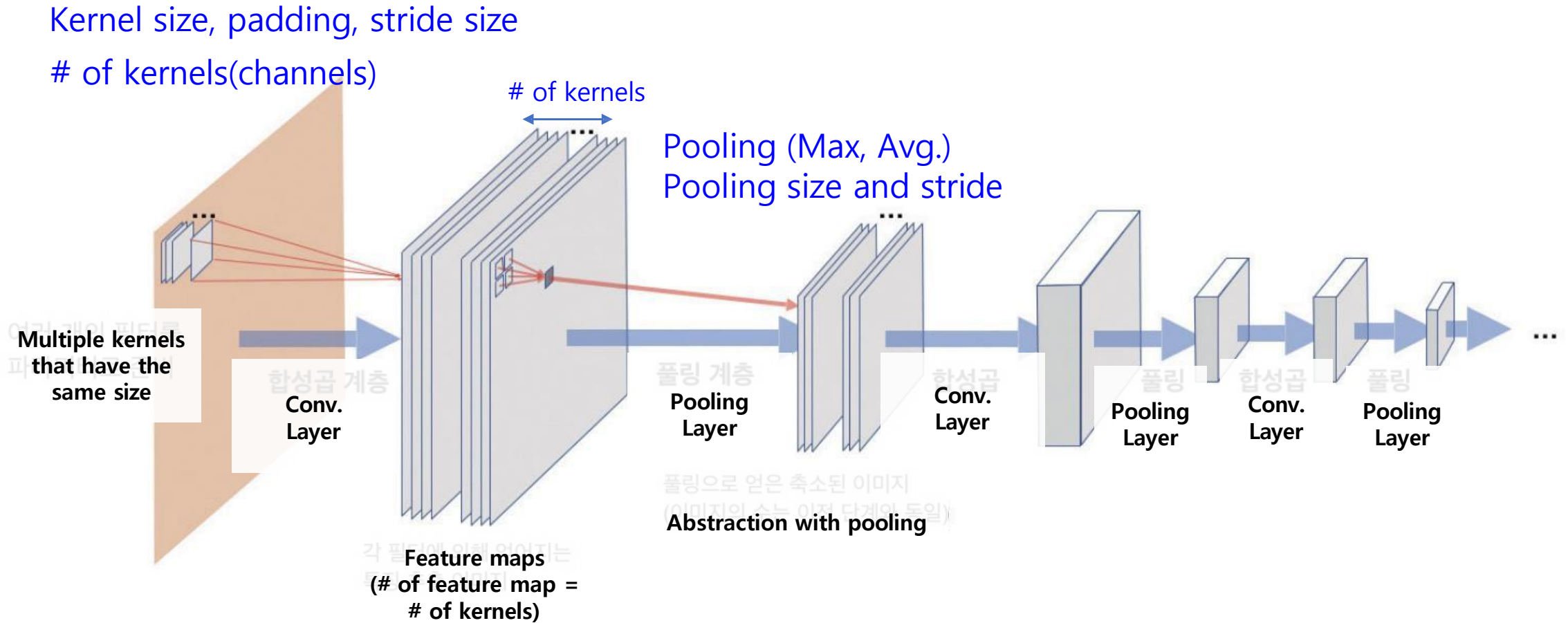


Abstraction with Pooling

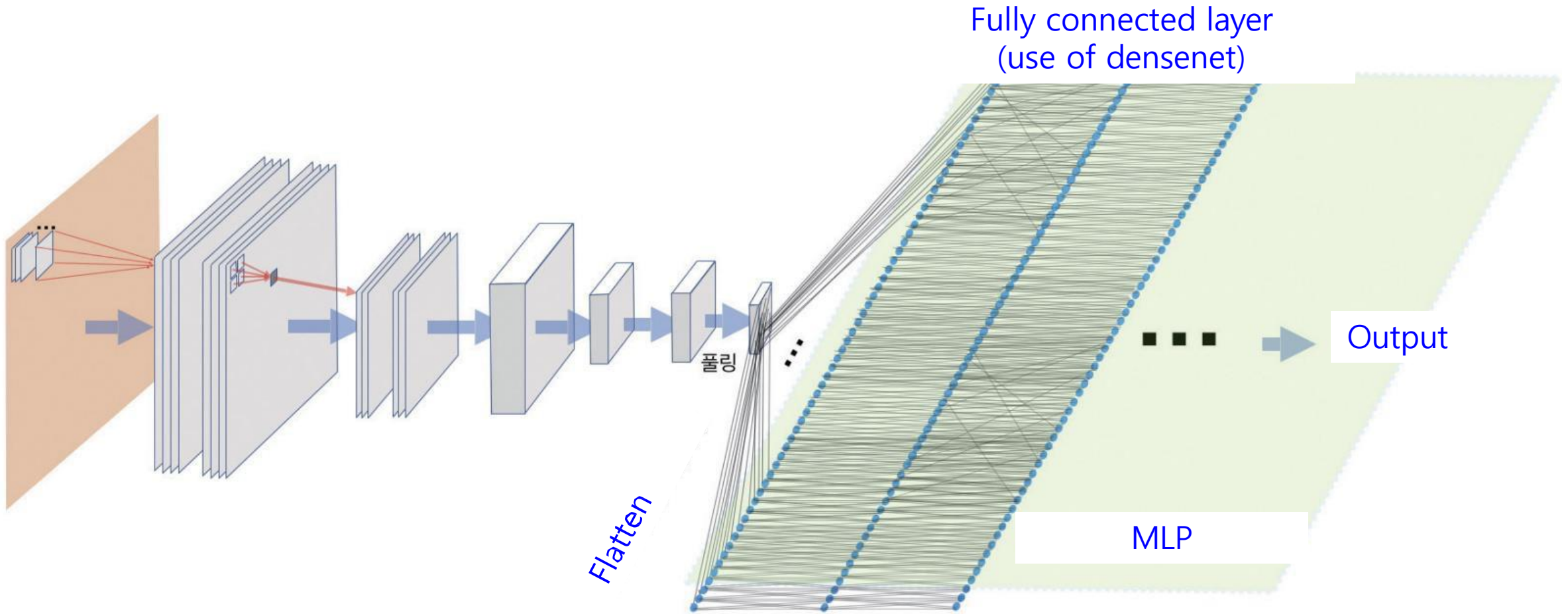
- Pooling is an operation that abstracts the values of pixels within a certain area of an image to a single value.
 - ✓ Max pooling: Selecting the largest value within the pooling area as the result.
 - ✓ Average pooling: taking the average value within the pooling area as the result



Convolution Layer + Pooling Layer



Flatten and connection to fully connected layer



Implementation of CNN (F-MNIST)



```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# 패션 MNIST 데이터를 학습용, 테스트 데이터로 구분하자
fashion_mnist = keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/train-labels-idx1-ubyte.gz
32768/29515 [=====] - 0s 0us/step
... <중간 생략> ...
```



```
train_images = train_images[:, :, :, np.newaxis]
test_images = test_images[:, :, :, np.newaxis]
train_images, test_images = train_images / 255, test_images / 255
```

Number of data instances, number of rows in the data, number of columns in the data, number of channels).

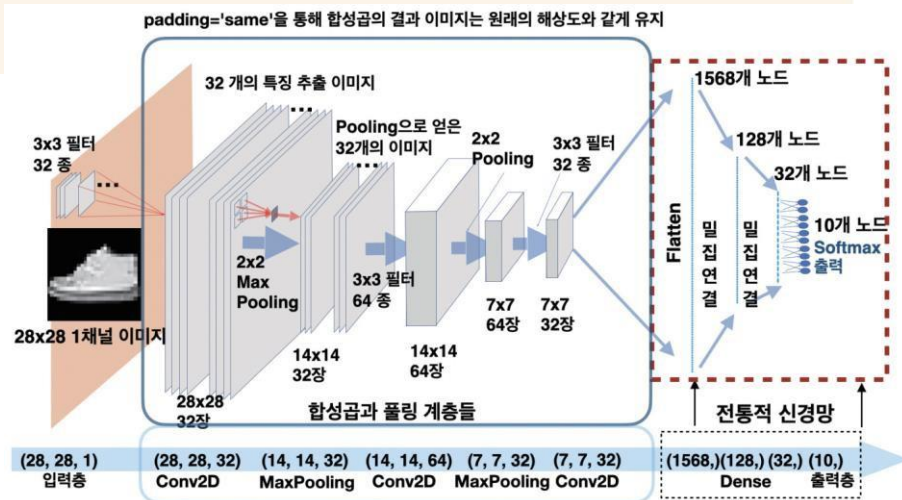


Implementation of CNN (F-MNIST)



```
model = keras.models.Sequential( [
    keras.layers.Conv2D(input_shape = (28, 28, 1),
                        kernel_size = (3,3), padding = 'same',
                        filters = 32),
    keras.layers.MaxPooling2D((2, 2), strides=2),
    keras.layers.Conv2D(kernel_size = (3,3), padding = 'same',
                        filters = 64),
    keras.layers.MaxPooling2D((2, 2), strides=2),
    keras.layers.Conv2D(kernel_size = (3,3), padding = 'same',
                        filters = 32),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation = 'relu'),
    keras.layers.Dense(32, activation = 'relu'),
    keras.layers.Dense(10, activation = 'softmax'),
])
```

model.summary()



Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
conv2d_3 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 14, 14, 64)	18496
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_5 (Conv2D)	(None, 7, 7, 32)	18464
flatten_1 (Flatten)	(None, 1568)	0
dense_3 (Dense)	(None, 128)	200832
dense_4 (Dense)	(None, 32)	4128
dense_5 (Dense)	(None, 10)	330
=====		

Total params: 242,570
 Trainable params: 242,570
 Non-trainable params: 0

Implementation of CNN (F-MNIST)



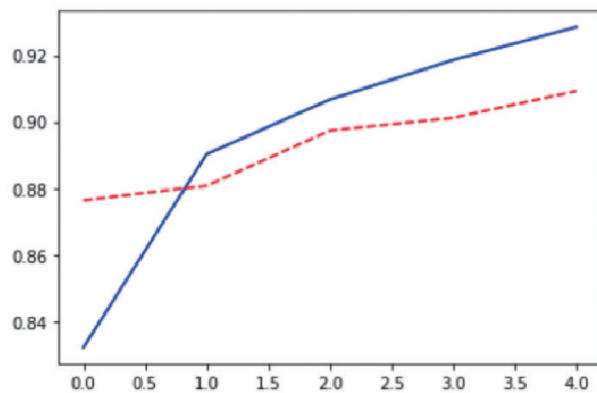
```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
hist = model.fit(train_images, train_labels,  
                epochs=5, validation_split=0.25)
```

Epoch 1/5

```
1407/1407 [=====] - 66s 46ms/step - loss: 0.4676 -  
... < 중간 생략 > ...  
1407/1407 [=====] - 64s 46ms/step - loss: 0.1934 -  
accuracy: 0.9284 - val_loss: 0.2602 - val_accuracy: 0.9092
```



```
plt.plot(hist.history['accuracy'], 'b-')  
plt.plot(hist.history['val_accuracy'], 'r--')
```



Implementation of CNN (F-MNIST)



```
test_loss, test_acc = model.evaluate(test_images, test_labels,\
                                     verbose=2)

print('테스트 정확도:', test_acc)
```

```
313/313 - 3s - loss: 0.2762 - accuracy: 0.9040
테스트 정확도: 0.9039999842643738
```



```
mnist_lbl = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat',
             'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle_boot']

images = test_images[:25]
pred = np.argmax(model.predict(images), axis=1)
print('예측값 =', pred)
print('실제값 =', test_labels[:25])
```

```
예측값 = [9 2 1 1 6 1 4 6 5 7 4 5 8 3 4 1 2 4 8 0 2 5 7 5 1]
실제값 = [9 2 1 1 6 1 4 6 5 7 4 5 7 3 4 1 2 4 8 0 2 5 7 9 1]
```

ImageNet

- What is ImageNet?
 - ImageNet is a large-scale, annotated database of millions of images designed for use in visual object recognition software research.
 - It has been a crucial dataset for training and benchmarking image recognition algorithms since its creation.
- The Significance of ImageNet:
 - Beyond its vast size, ImageNet is structured according to the WordNet hierarchy, which organizes images into thousands of categories. This structure has enabled more nuanced and accurate image classification tasks.
 - The annual ImageNet Large Scale Visual Recognition Challenge (ILSVRC) started in 2010, pushing the boundaries of algorithmic accuracy and speed in image classification and object detection.



- ImageNet contains 1,281,167 training images
- ImageNet contains 50,000 validation images
- ImageNet contains 100,000 test images
- ImageNet contains 1000 object classes

ImageNet

- Impact on CNN Development:
 - The challenges presented by ImageNet have driven significant breakthroughs in CNN architectures. Winning models like AlexNet, VGG, and ResNet showcased revolutionary improvements in handling complex image recognition tasks.
 - The competitive environment fostered by the ILSVRC has accelerated the development of more efficient, accurate, and deeper neural networks, influencing not just academic research but also practical applications in industry.
- Beyond ImageNet:
 - While ImageNet has played a pivotal role in the progress of image recognition, the field continues to evolve with new datasets and challenges that address limitations such as dataset bias and the need for more complex object interaction understanding.
 - The legacy of ImageNet underscores the importance of large, well-annotated datasets in advancing machine learning and artificial intelligence technologies.

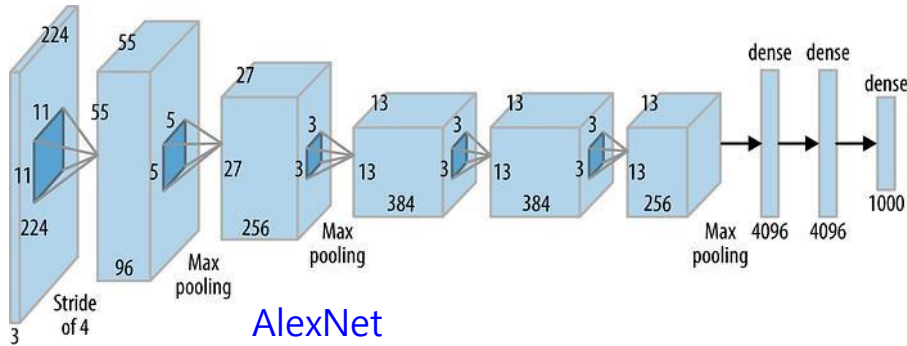
Core Components of CNN for Image Recognition

- Key Components of Convolutional Neural Networks (CNNs):
 - Convolutional Layer: The cornerstone layer of CNNs, responsible for extracting features from the input image using filters (or kernels) to learn local patterns.
 - Activation Function: Introduces non-linearity into the network, allowing it to learn complex patterns. The Rectified Linear Unit (ReLU) is a commonly used activation function.
 - Pooling Layer: Reduces the spatial size of the representation, decreasing the number of parameters and computation in the network. Max pooling is frequently utilized to reduce sensitivity to the exact location of features.
- Basic Working Principle of CNNs:
 - CNNs progressively extract higher-level features from the input image by passing it through multiple convolutional and pooling layers, culminating in a classifier that performs tasks such as image classification.
- Importance of Feature Maps:
 - Feature maps generated at each layer contain important information about the image, becoming increasingly refined as the network learns during training.
- Fully Connected Layer for Final Classification:
 - At the end of the network, fully connected layers utilize all extracted features to make the final classification decision, identifying the class to which the image belongs.

CNN models

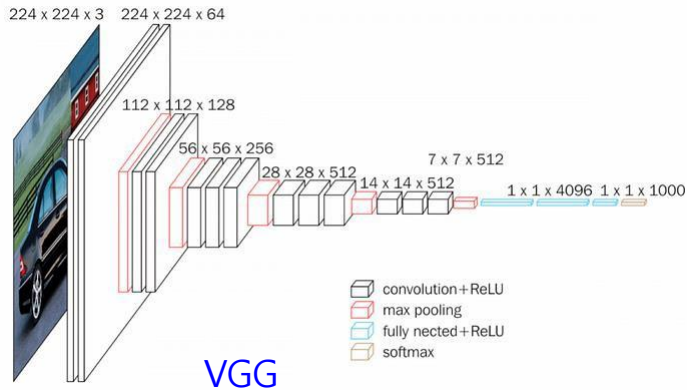
AlexNet

ReLU,
데이터증강,
Dropout



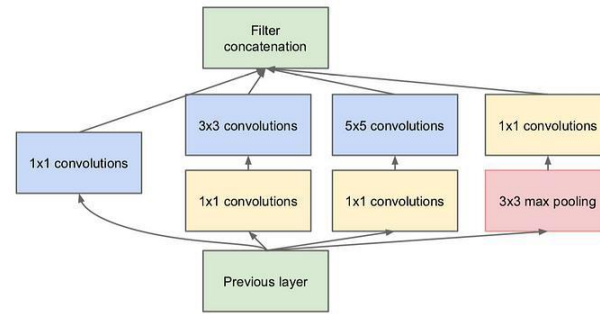
VGG

깊은 층
MaxPooling



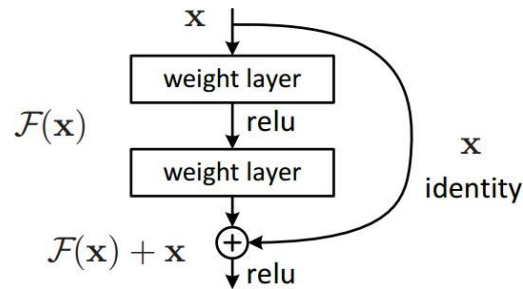
Inception

인셉션 모듈 구조
계산효율성과 성능



ResNet

잔차 학습의 개념
깊은 네트워크 학습
문제 해결



DenseNet

밀집 연결 구조
효율적인 특징 재상
용

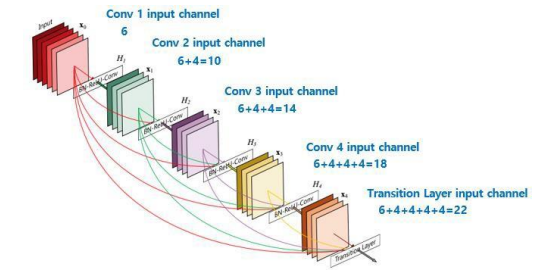
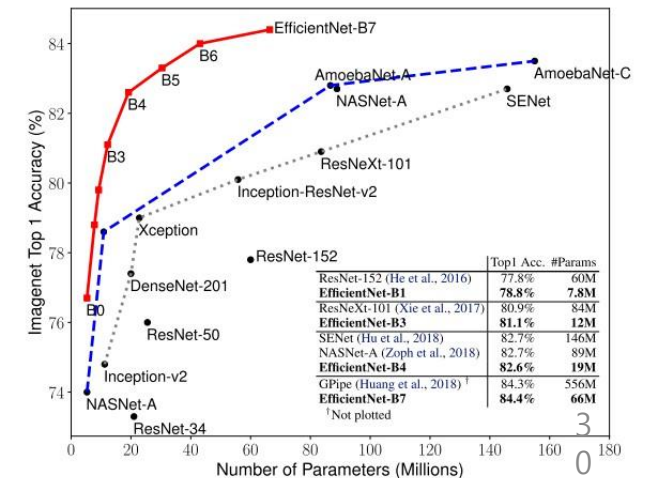


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

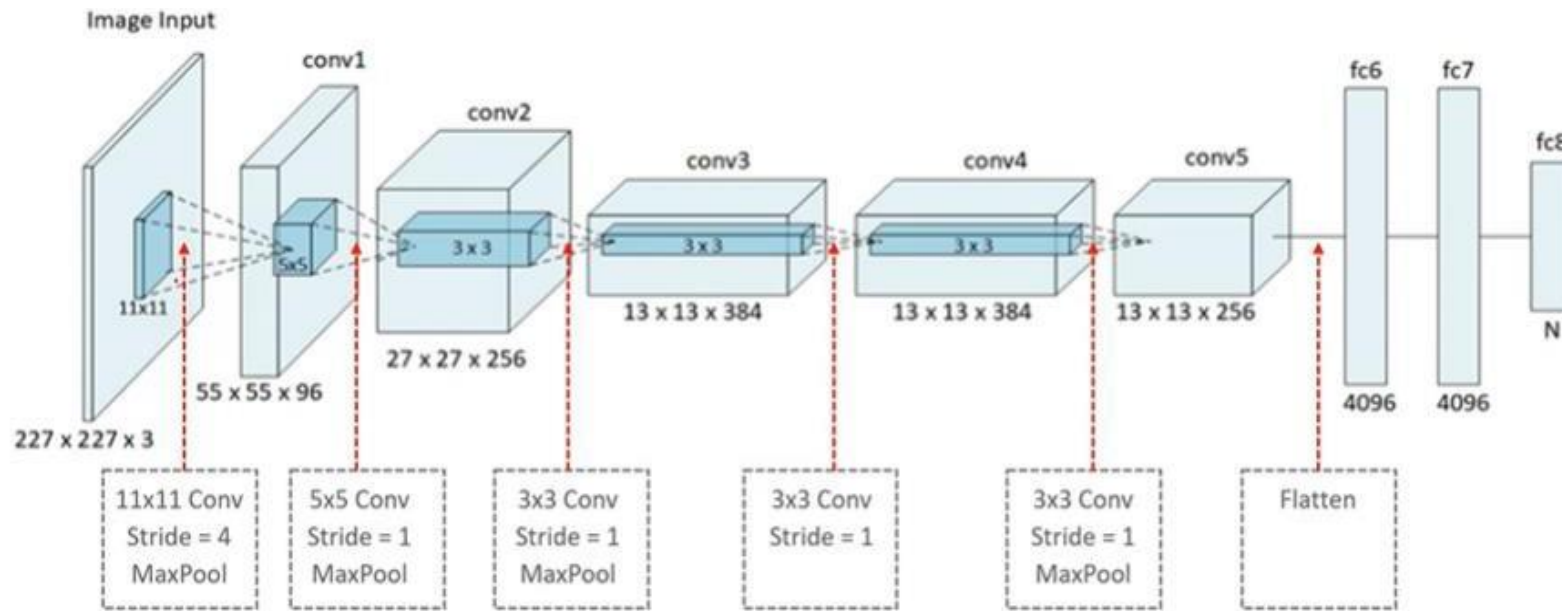
EfficientNet

복잡도 조절을 통한
스케일링



01_AlexNet

- Structure:
 - AlexNet consists of five convolutional layers followed by three fully connected layers. It was the first to use the ReLU (Rectified Linear Unit) activation function extensively across its layers, promoting faster training without the vanishing gradient problem.



- 11x11, 5x5 사이즈의 큰 사이즈의 Kernel 적용. 이후 3x3 Kernel을 3번 이어서 적용
- Receptive Field가 큰 사이즈를 초기 Feature map에 적용하는 것이 보다 많은 feature 정보를 만드는데 효율적이라고 판단
- 하지만 많은 weight parameter 갯수로 인하여 컴퓨팅 연산량이 크게 증가 함.

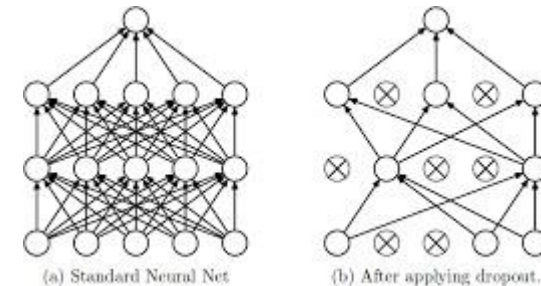
01_AlexNet

- Innovations:
 - Dropout layers to combat overfitting,
 - Overlapping pooling to reduce the size of the network while retaining important information
 - Data augmentation techniques to enhance the generalizability of the model.

Dropout 작동 방식

- 훈련 시, 네트워크의 각 레이어마다 설정된 확률에 따라 일부 뉴런(노드)를 임의로 비활성화(즉, 출력을 0으로 설정)
- 비활성화된 뉴런은 그 레이어의 전파 과정과 역전파 과정에서 제외(다양한 뉴런의 조합을 통해 데이터를 학습)
- 테스트 시에는 모든 뉴런이 활성화되지만, 각 뉴런의 출력은 훈련 시 비활성화됐던 비율을 고려하여 조정(훈련 과정에서의 뉴런의 비활성화가 모델의 평균적인 예측에 영향을 주지 않도록 보정하는 역할)

효과: 과적합 방지, 모델의 강건성 향상, 앙상블 효과

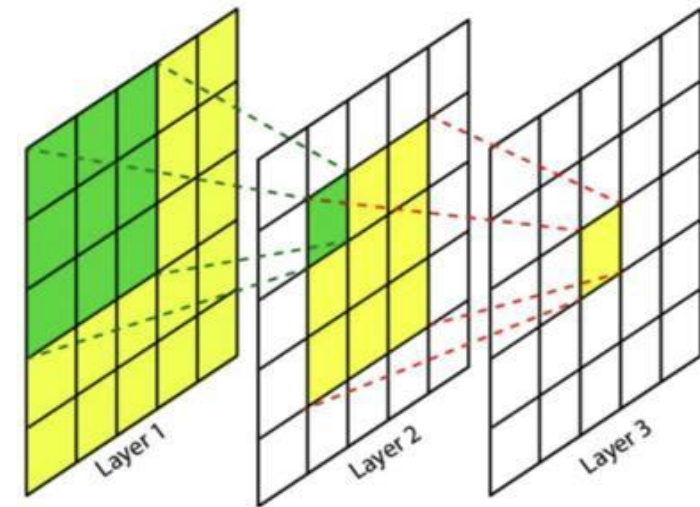


02_VGG

- Uniform Convolutional Layers: VGGNet is known for its simplicity, utilizing 3x3 convolutional filters uniformly across the network. This approach, combined with the increased depth of up to 19 layers (VGG-19 being the most famous), enables it to capture complex features at various levels.
- Stacking Convolutional Layers: By stacking multiple 3x3 convolutional layers, VGGNet simulates the effect of larger receptive fields (such as 5x5 or 7x7 filters) but with fewer parameters and lower computational cost.



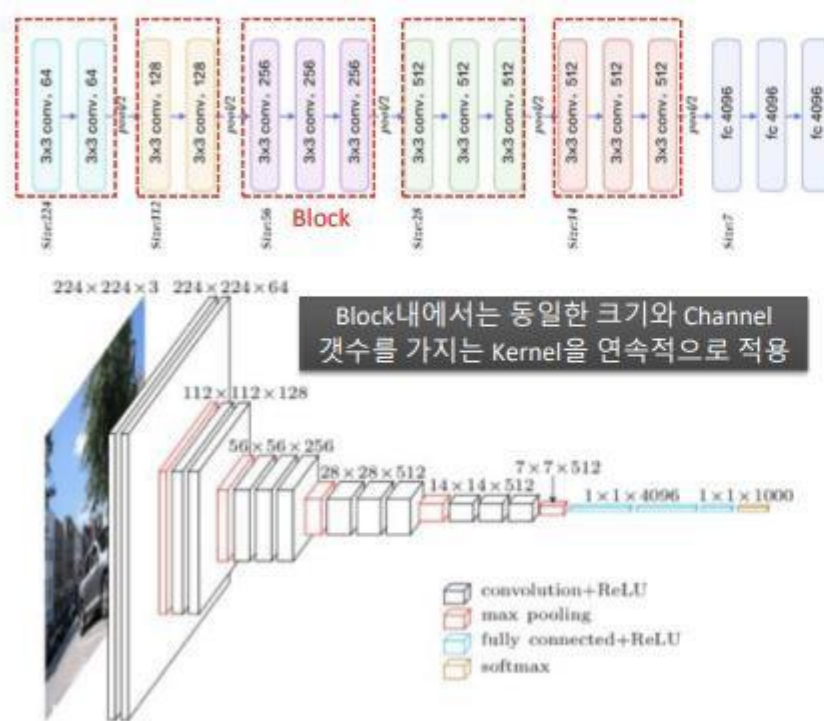
VGG구조



Stacking multiple 3x3 convolutional layers

02_VGG

- Pooling Layers: Max pooling is used between convolutional layers to reduce feature map dimensions and to prepare for the next set of convolutional operations, further refining the network's ability to process visual information.



- 11X11, 5X5 와 같은 넓은 크기의 Kernel로 Convolution 연산을 적용하는 것보다 여러 개의 3X3 Convolution 연산을 수행하는 것이 더 뛰어난 Feature 추출 효과를 나타냄.
- AlexNet 대비 더 많은 채널수와 더 깊은 Layer 구성.
- 3x3 크기의 Kernel을 (2번 또는 3번)연속해서 Convolution 적용한 뒤에 Max Pooling 적용하여 일련의 Convolution Feature map Block을 생성
- 개별 Block내에서는 동일한 커널 크기와 Channel 갯수를 적용하여 동일한 크기의 feature map들을 생성
- 이전 Block 내에 있는 Feature Map 대비 새로운 Block내에 Feature Map 크기는 2배로 줄어 들지만 채널 수는 2배로 늘어남.(맨 마지막 block 제외)

02_VGG

VGG16 - Structural Details													
#	Input Image			output			Layer	Stride	Kernel		in	out	Param
1	224	224	3	224	224	64	conv3-64	1	3	3	3	64	1792
2	224	224	64	224	224	64	conv3-64	1	3	3	64	64	36928
	224	224	64	112	112	64	maxpool	2	2	2	64	64	0
3	112	112	64	112	112	128	conv3-128	1	3	3	64	128	73856
4	112	112	128	112	112	128	conv3-128	1	3	3	128	128	147584
	112	112	128	56	56	128	maxpool	2	2	2	128	128	65664
5	56	56	128	56	56	256	conv3-256	1	3	3	128	256	295168
6	56	56	256	56	56	256	conv3-256	1	3	3	256	256	590080
7	56	56	256	56	56	256	conv3-256	1	3	3	256	256	590080
	56	56	256	28	28	256	maxpool	2	2	2	256	256	0
8	28	28	256	28	28	512	conv3-512	1	3	3	256	512	1180160
9	28	28	512	28	28	512	conv3-512	1	3	3	512	512	2359808
10	28	28	512	28	28	512	conv3-512	1	3	3	512	512	2359808
	28	28	512	14	14	512	maxpool	2	2	2	512	512	0
11	14	14	512	14	14	512	conv3-512	1	3	3	512	512	2359808
12	14	14	512	14	14	512	conv3-512	1	3	3	512	512	2359808
13	14	14	512	14	14	512	conv3-512	1	3	3	512	512	2359808
	14	14	512	7	7	512	maxpool	2	2	2	512	512	0
14	1	1	25088	1	1	4096	fc		1	1	25088	4096	102764544
15	1	1	4096	1	1	4096	fc		1	1	4096	4096	16781312
16	1	1	4096	1	1	1000	fc		1	1	4096	1000	4097000
Total												138,423,208	